

Объектно-ориентированное программирование в Python

Содержание урока

- ★ ООП. Основные принципы
- ★ Классы и объекты
- ★ Методы
- ★ Конструкторы и деструкторы
- ★ Инкапсуляция
- ★ Наследование
- ★ Полиморфизм



ВАСИЛИСА СЕРЯКОВА

Data Scientist @Aliexpress

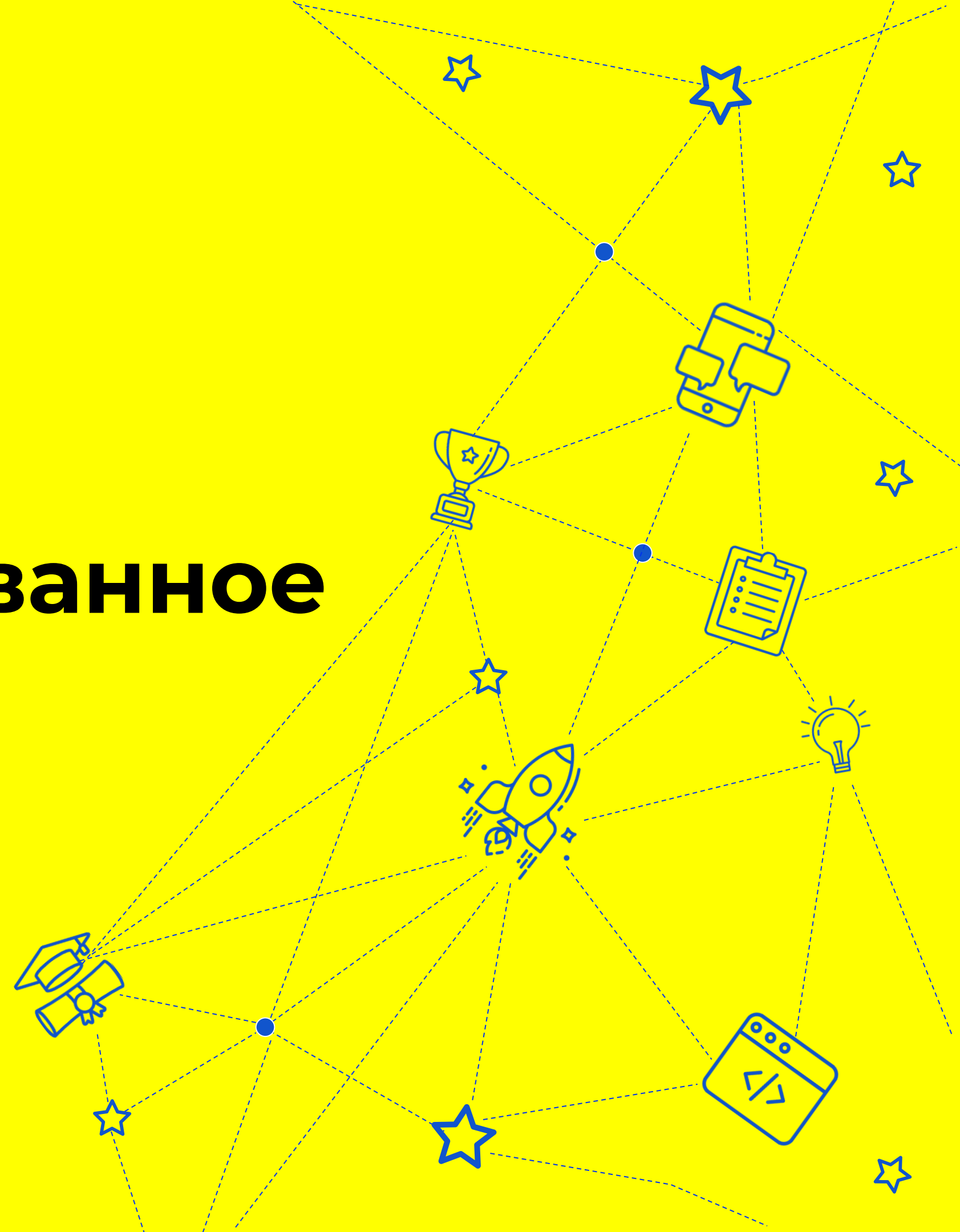
ex-Data Scientist @Yandex



Яндекс  Маркет

- Делаю машинное обучение для рекламы в **Aliexpress**
- Занималась **Data Engineering** в **Blockchain.com**, строила пайплайны для автоматической проверки транзакций
- Почти 3 года делала **Матчинг** в **Яндекс.Маркете**
- 1 год преподавала в **НИУ ВШЭ** курсы по **Python** и **Машинному обучению**

Объектно-ориентированное программирование



ООП – зачем?

• Основная суть ООП

— это представление программы в виде набора объектов, которые могут взаимодействовать между собой и внешним миром.

Взаимодействие:



Создание одного объекта другим



Удаление объекта



Обмен сообщениями

Классы

•Класс

— это тип данных, с помощью которого создаются объекты в ООП.

Класс можно представить как некоторый чертеж или инструкцию, по которой можно создавать нужные нам объекты.

```
class A: # описание класса
    def __init__(self, a): # конструктор класса
        self.a = a # поля класса

b = A(3) # создание объекта из класса
print(b.a) # 3
```

Классы

Класс состоит из данных и функций, работающих с этими данными.

Эти функции, привязанные к конкретному классу, называются **методами**, а данные класса **полями**.

```
class A: # описание класса
    def __init__(self, a): # конструктор класса
        self.a = a # поля класса

b = A(3) # создание объекта из класса
print(b.a) # 3
```

Конструктор класса

За создание объекта класса отвечает конструктор.

• Конструктор

— это особый метод который вызывается при создании объекта.

```
class A: # описание класса
    def __init__(self, a): # конструктор класса
        self.a = a # поля класса

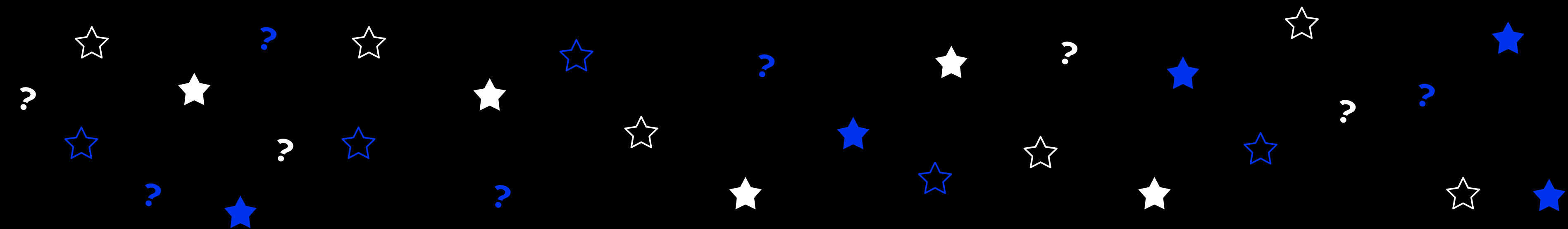
b = A(3) # создание объекта из класса
print(b.a) # 3
```

• в Python
– это метод

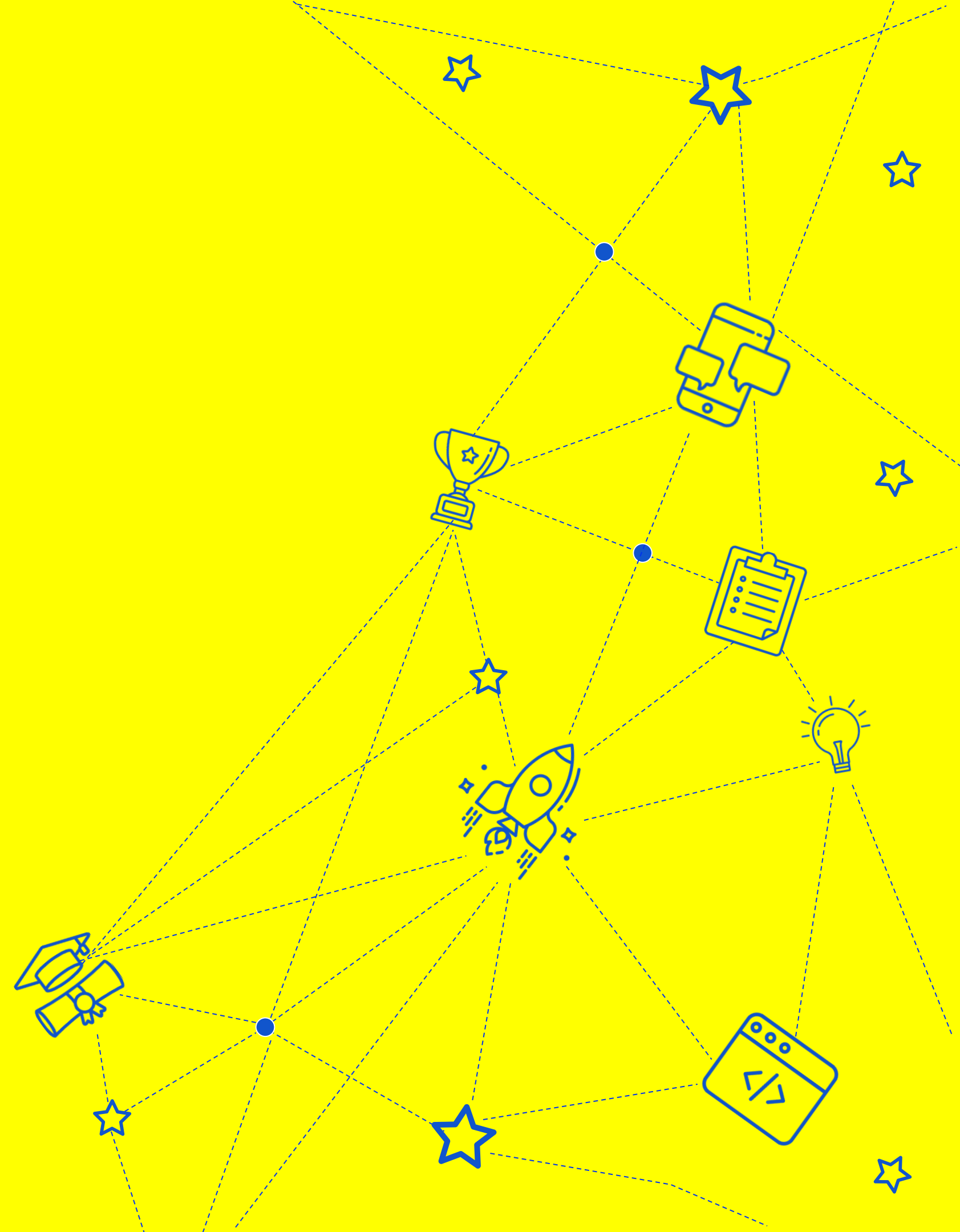
`__init__()`

Спасибо за внимание

Продолжение в следующем видео



Классы и объекты в Python



Классы и объекты в Python

В Python все является объектом какого-то класса, даже примитивные типы данных, функции и сами классы!

```
1 print(type(3)) # тип объекта целое
2 print(type("строка")) # тип объекта строка
3 print("строка".upper()) # вызов метода строки
```

```
<class 'int'>
<class 'str'>
СТРОКА
```

Например строки имеют набор методов

Классы и объекты в Python

```
1 class Car(Vehicle): # имя класса (класс предок)
2     wheels_count = 4 # поле или атрибут класса
3     is_motor = True # поле или атрибут класса
4     is_run = False
5
6     def drive(self): # метод класса
7         self.is_run = True
8         print("we a run")
```

• Определение класса

Классы и объекты в Python

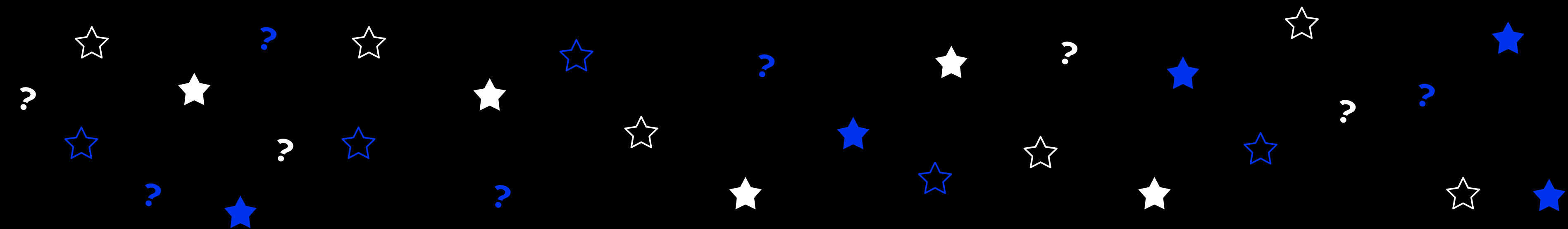
```
4 class Car(Vehicle): # имя класса ( класс предок)
5     wheels_count = 4 # поле или атрибут класса
6     is_motor = True # поле или атрибут класса
7     is_run = False
8
9     def drive(self): # метод класса
10         self.is_run = True
11         print("we a run")
12
13 myCar = Car() # создание объекта
14 myCar.drive() # вызов метода
```

we a run

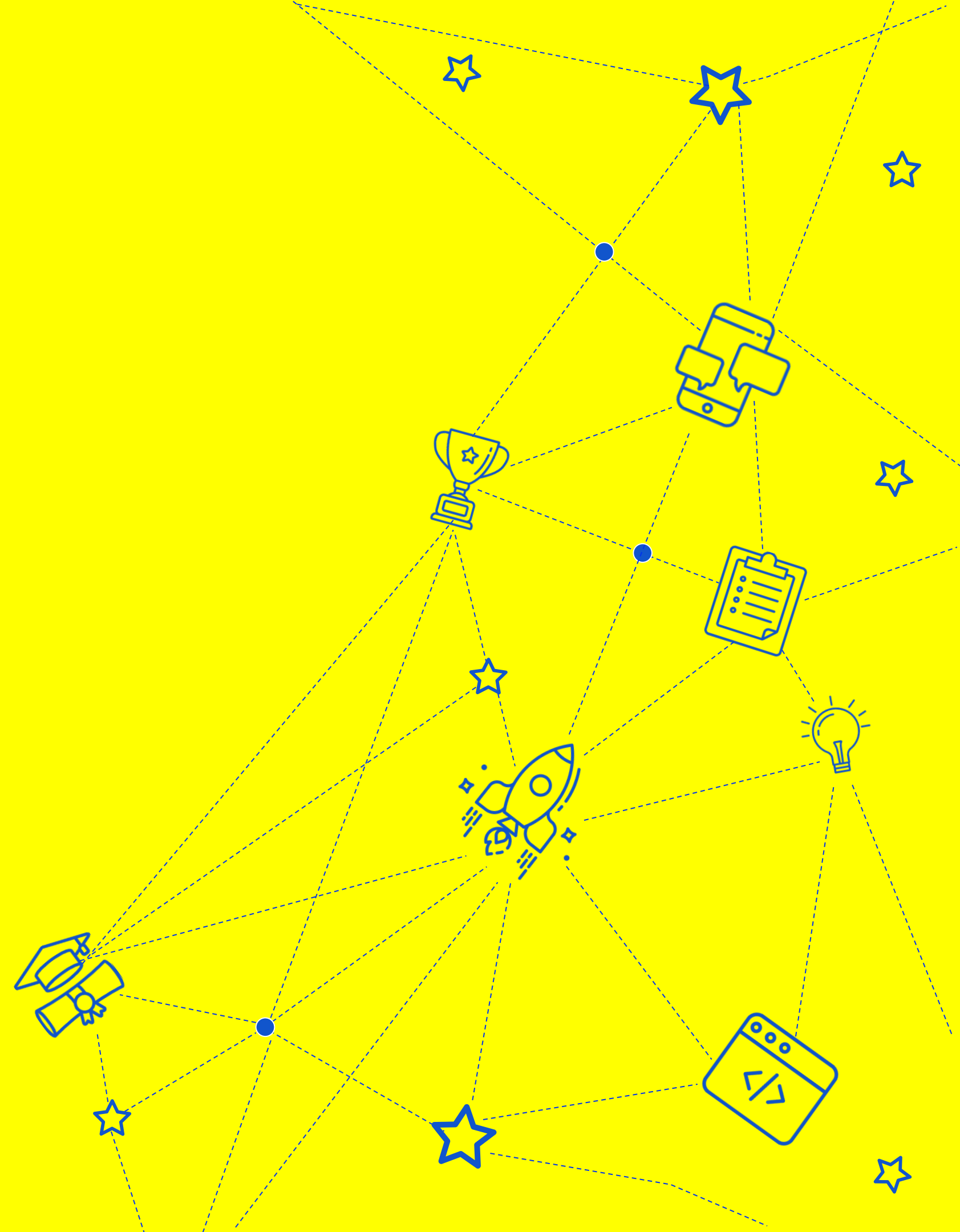
Создание объекта

Спасибо за внимание

Продолжение в следующем видео



Методы



Методы

• Методы объектов

— функции, вызываемые для объекта, экземпляра класса.

```
1 class Car(Vehicle): # имя класса (класс предок)
2     wheels_count = 4 # поле или атрибут класса
3     is_motor = True # поле или атрибут класса
4     is_run = False
5
6     def drive(self): # метод класса
7         self.is_run = True
8         print("we a run")
```

• Ссылка на объект, для которого они вызваны передается как

self

Методы

• Методы класса

— функции, вызываемые для класса

Такие методы помечаются специальным декоратором



@classmethod

Метод класса принимает в качестве первого аргумента ссылку на класс **(не объект!!!)**.

Методы класса могут менять состояние класса, что отражается на всех объектах класса

Методы

• Статические методы

— функции, привязанные к классу логически, но не имеющие доступа ни к объекту, ни к атрибутам класса.

Такие методы помечаются специальным декоратором

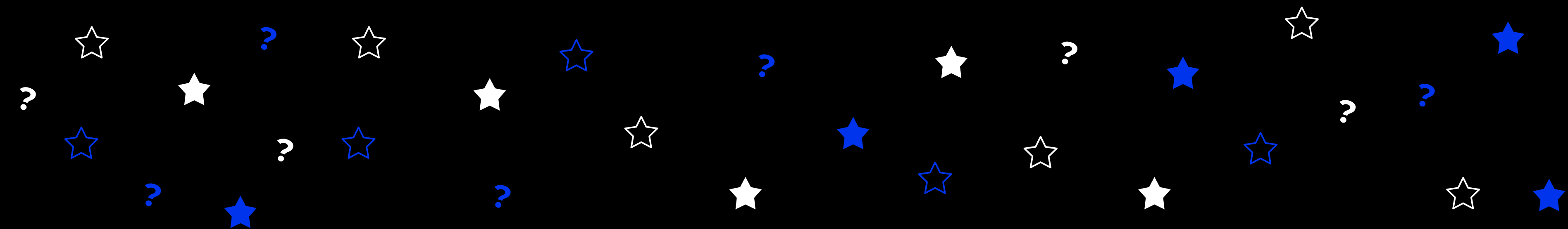


@staticmethod

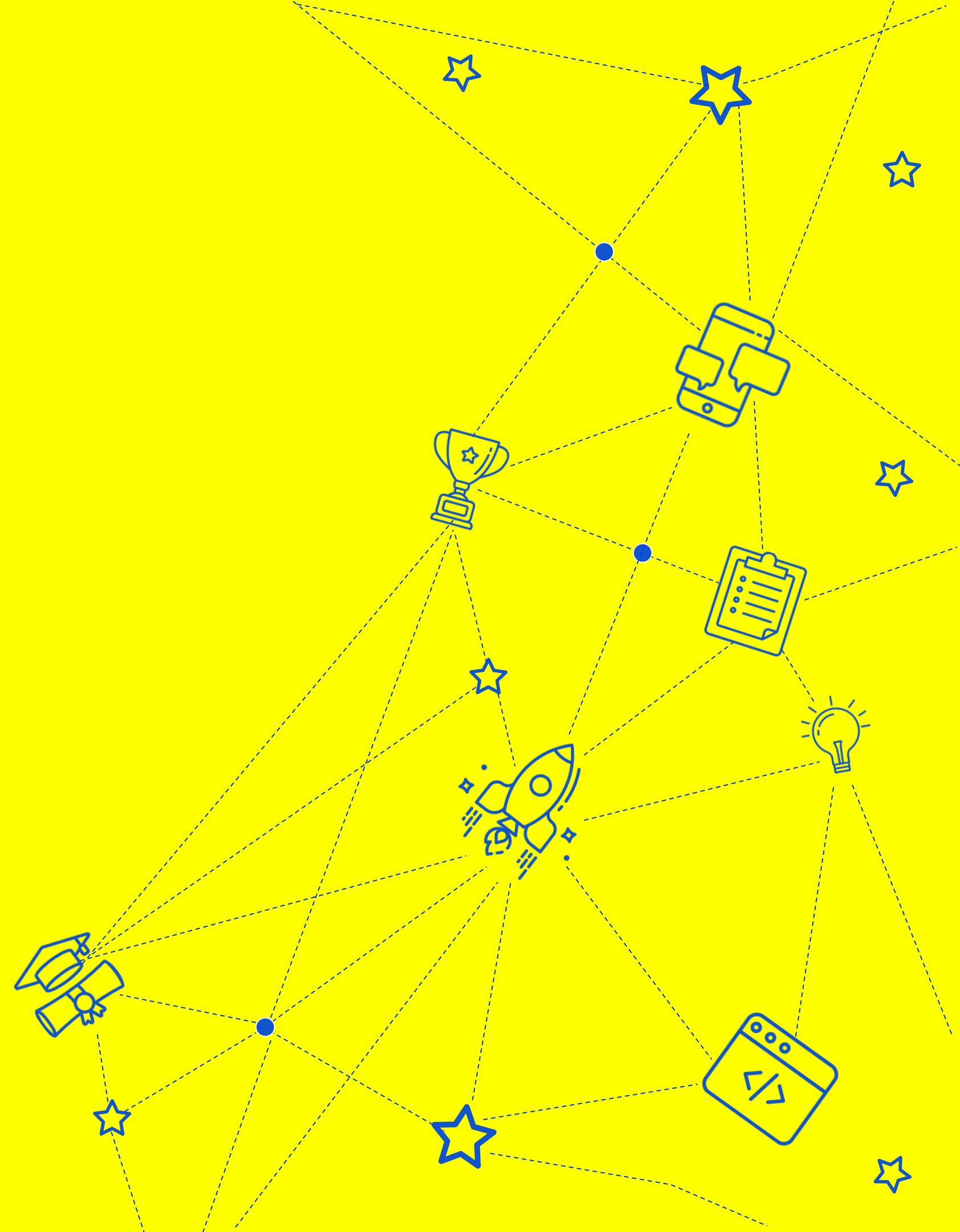
Статические методы также называют методами
“не знаю, к какому классу отношусь”

Спасибо за внимание

Продолжение в следующем видео



Конструкторы и деструкторы



Конструкторы

Особый метод

```
def __init__(self):  
    # some  
    work
```

Конструктор будет вызван при создании класса автоматически.
Аргументы передаются в конструктор из вызова класса.

Конструкторы

```
1 class Car(Vehicle):
2     def __init__(self, serialID):
3         self.sID = serialID
4         print(f"car {serialID} created")
5
6 car = Car(1)
```

car 1 created

```
1 class Car(Vehicle):
2     def init(self, serialID):
3         self.sID = serialID
4         print(f"car {serialID} created")
5
6 car = Car()
7 car.init(1)
```

car 1 created

Деструкторы

- Противоположный по смыслу конструктору метод

```
def __del__():  
    # some  
    work
```

- Вызывается при удалении объекта

На практике используется гораздо реже, чем конструктор.

Применяется для закрытия связанных с объектом файлов, сетевых подключений.

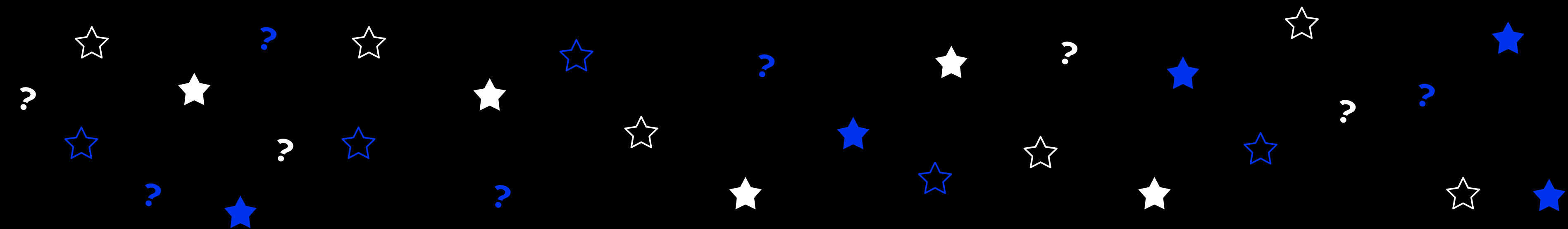
Деструкторы

```
1 class Car(Vehicle):
2     def __init__(self, serialID):
3         self.sID = serialID
4         print(f"car {serialID} created")
5
6     def __del__(self):
7         print("car destroyed")
8
9 car = Car(1)
10 del car
11
```

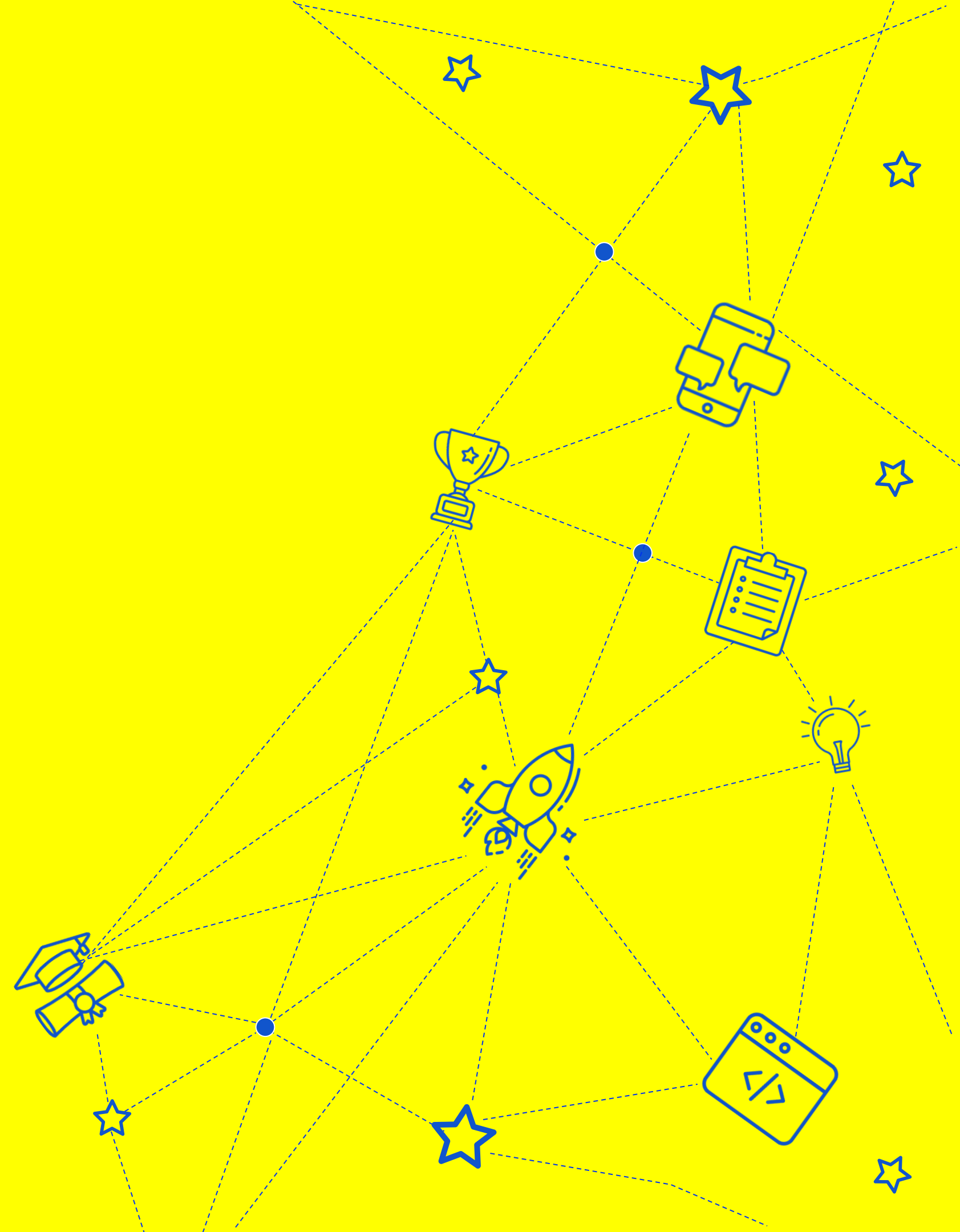
```
car 1 created
car destroyed
```


Спасибо за внимание

Продолжение в следующем видео



Инкапсуляция



Инкапсуляция

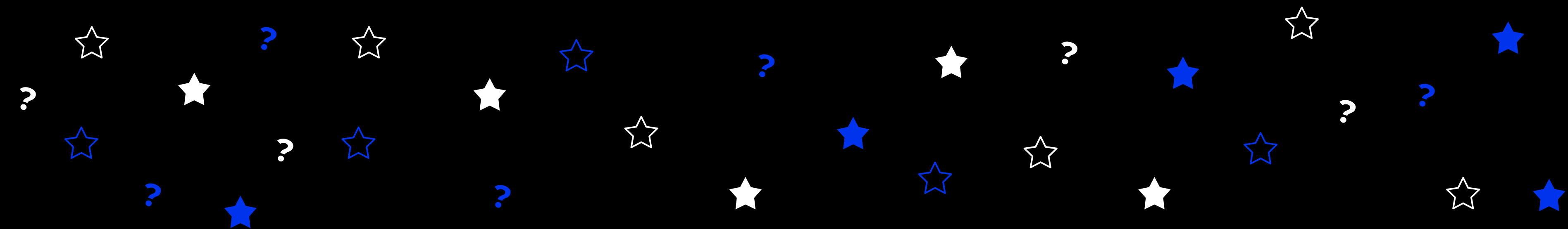
• Инкапсуляция

— это связывание в одном объекте данных и поведения, то есть методов работы с этими данными.

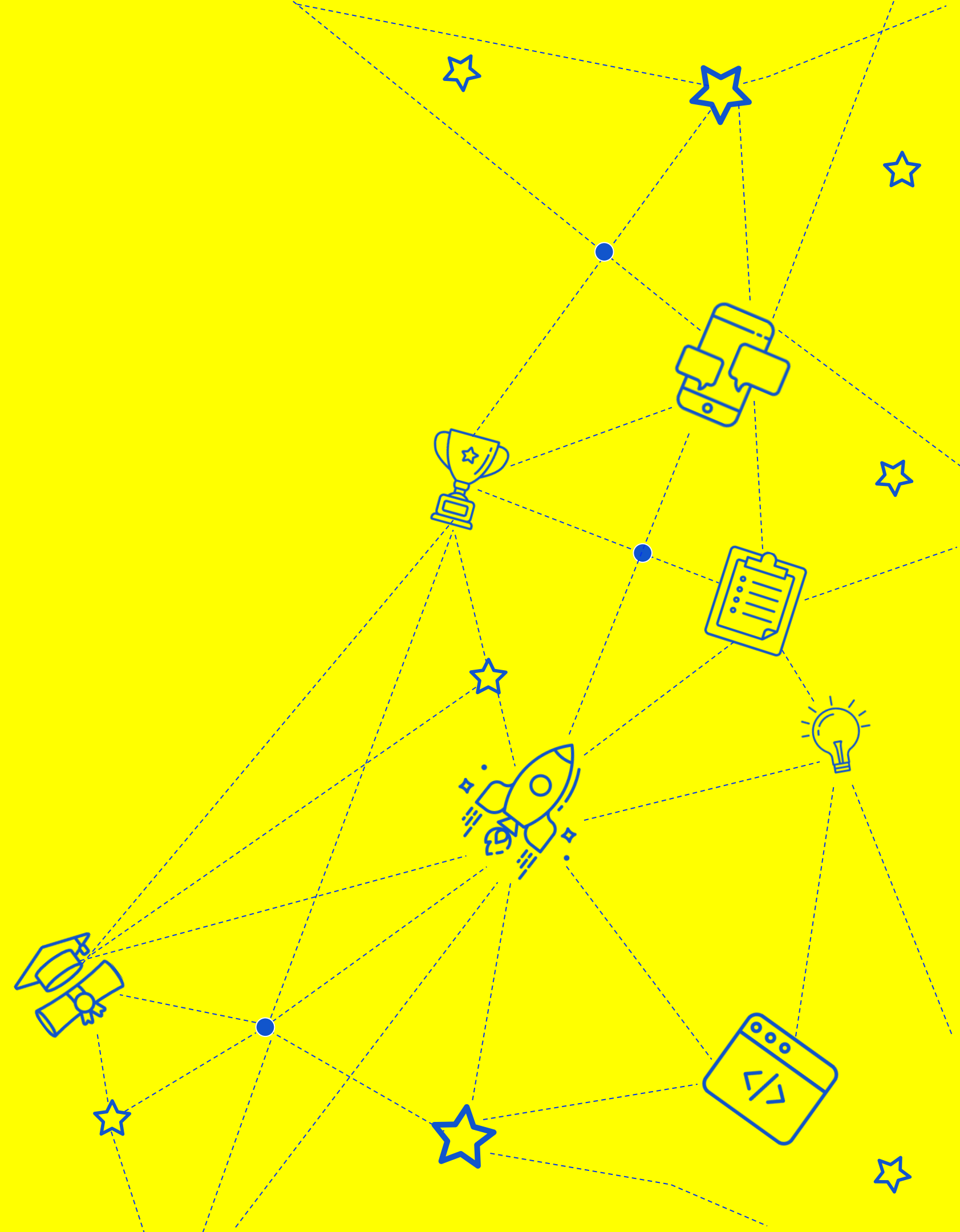
```
1 class A: # описание класса
2     def __init__(self, a): # конструктор класса
3         self.a = a # поля класса
4     def show(self): # метод объекта
5         print(self.a) #self -ссылка на объект
6     def add(self, b):
7         self.a += b
8
9 b = A(3) # создание объекта из класса
10 b.show() # 3
11 b.add(5) #
12 b.show() # 8
```

Спасибо за внимание

Продолжение в следующем видео



Наследование



Наследование

• Наследование

— это расширение одного класса другим.

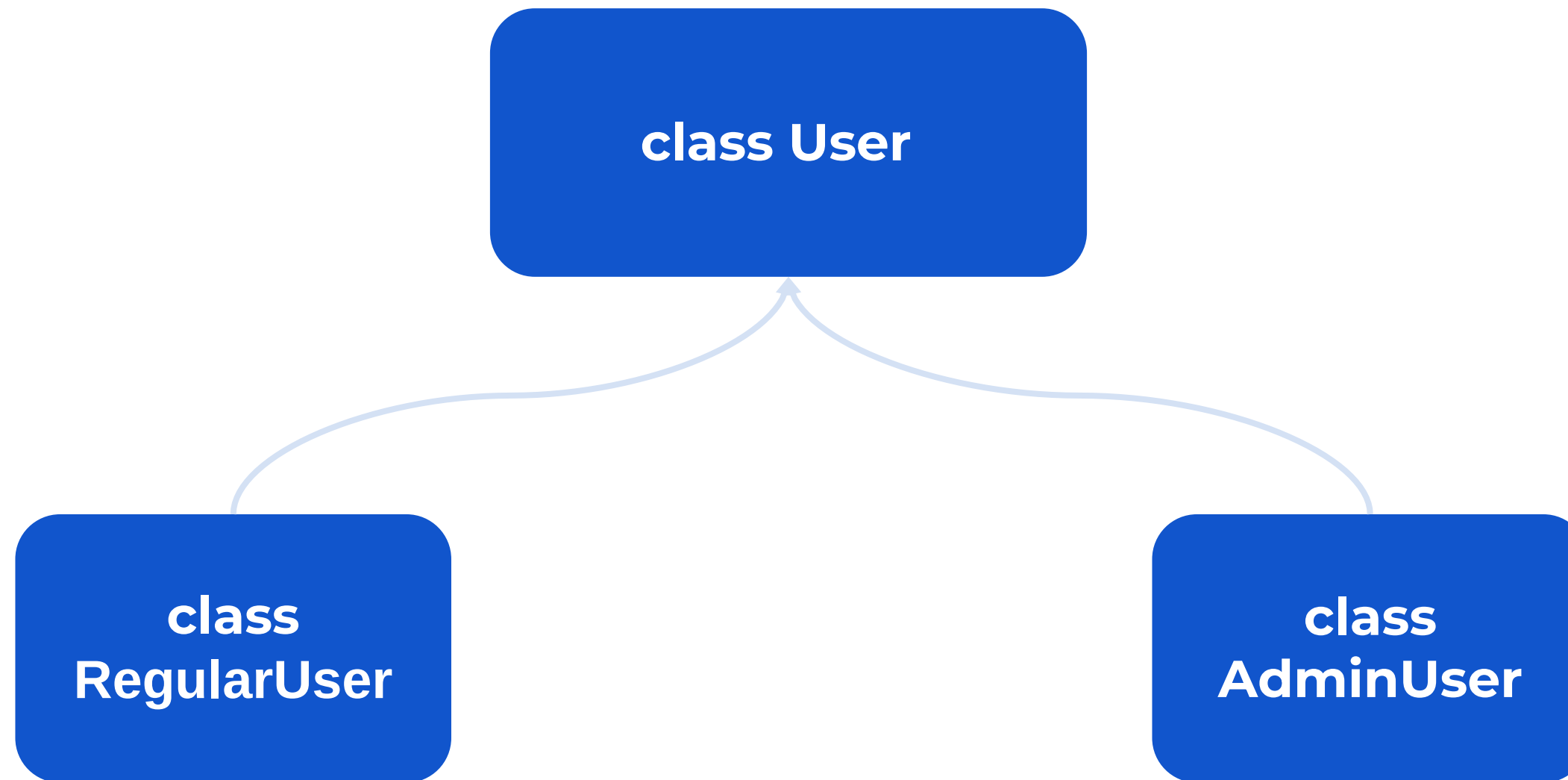
Класс, от которого наследуют, называется **родительским или базовым**.

Класс, который наследует другой класс, называют **дочерним или производным**.

Направление идет от более высоких абстракций к более конкретным.

Наследование

Пример



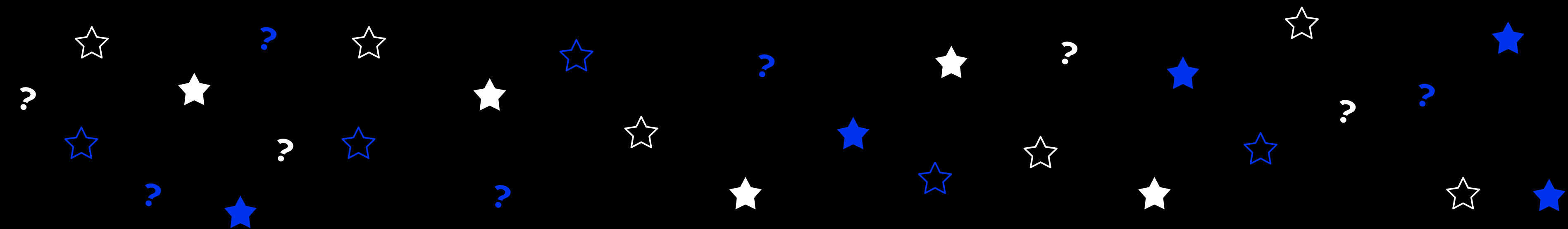
Наследование

Пример

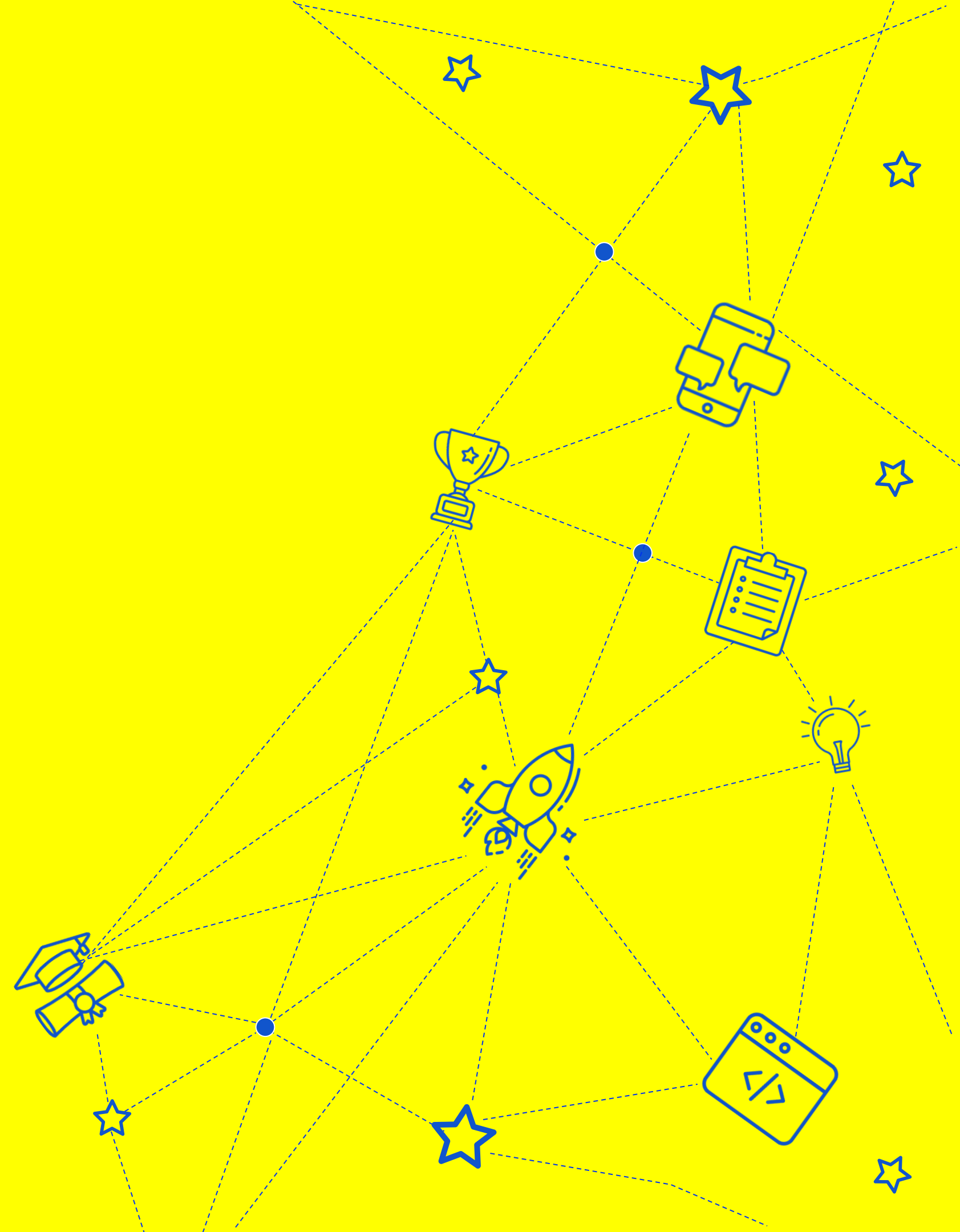
```
1 class User:
2     name = ""
3     email = ""
4     approved_by = ""
5     def __init__(self, name, email):
6         self.name = name
7         self.email = email
8     def show(self):
9         print(f"Name: {self.name} Email: {self.email} ApprovedBy: {self.approved_by}")
10
11 class AdminUser(User):
12     def __init__(self, name, email):
13         super().__init__(name, email) # super вернет класс - родитель от которого мы и унаследуемся
14         self.approved_by = "admin"
15     def approve(self, user : User):
16         user.approved_by = self.name
17
18
19 user = User("aleks", "mail@alex.com")
20 admin = AdminUser("ivan", "mail@ivan.com")
21 admin.approve(user)
22 user.show()
23 admin.show() # этот метод мы унаследовали от User
```


Спасибо за внимание

Продолжение в следующем видео



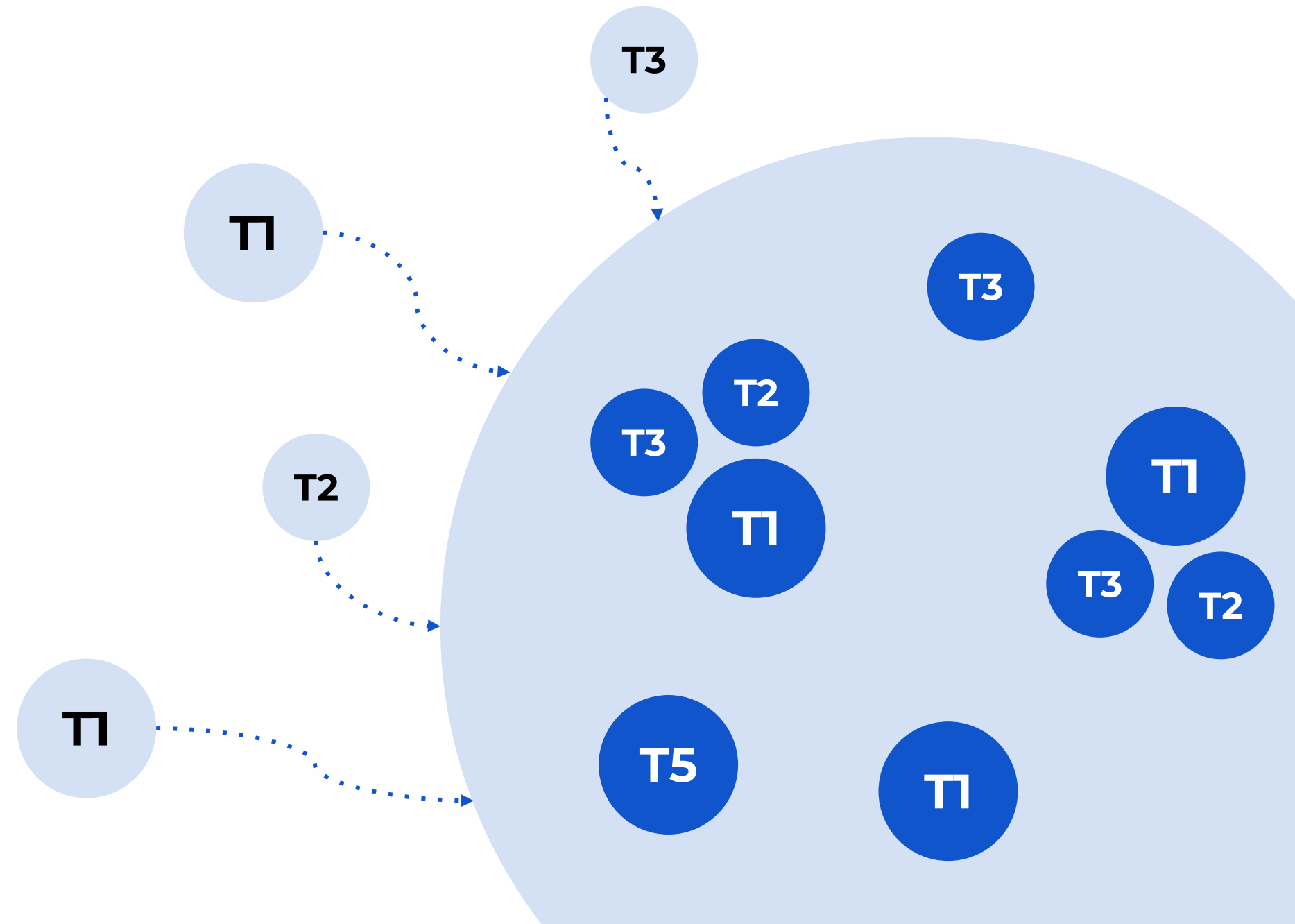
Полиморфизм



Полиморфизм

• Полиморфизм в ООП

— это способность функции принимать объекты различных типов и работать с ними обобщенно



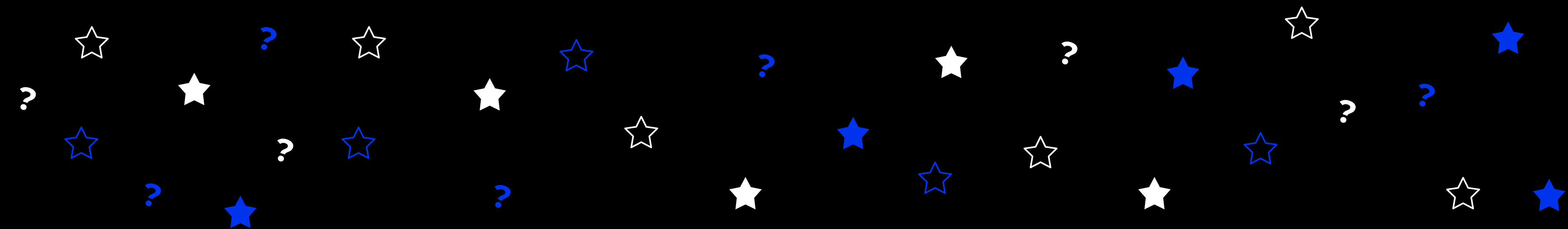
Полиморфизм

```
1 class User:
2     name = ""
3     email = ""
4     approved_by = ""
5     def __init__(self, name, email):
6         self.name = name
7         self.email = email
8     def show(self):
9         print(f"Name: {self.name} Email: {self.email} ApprovedBy: {self.approved_by}")
10
11 class AdminUser(User):
12     def __init__(self, name, email):
13         super().__init__(name, email) # super вернет класс - родитель от которого мы и унаследуемся
14         self.approved_by = "admin"
15     def approve(self, user : User):
16         user.approved_by = self.name
17
18
19 user = User("aleks", "mail@alex.com")
20 admin = AdminUser("ivan", "mail@ivan.com")
21 admin.approve(user)
22 user.show()
23 admin.show() # этот метод мы унаследовали от User
24
25
26 admin2 = AdminUser("petr", "mail@petr.com")
27 admin2.approve(admin)
28 admin.show()
```

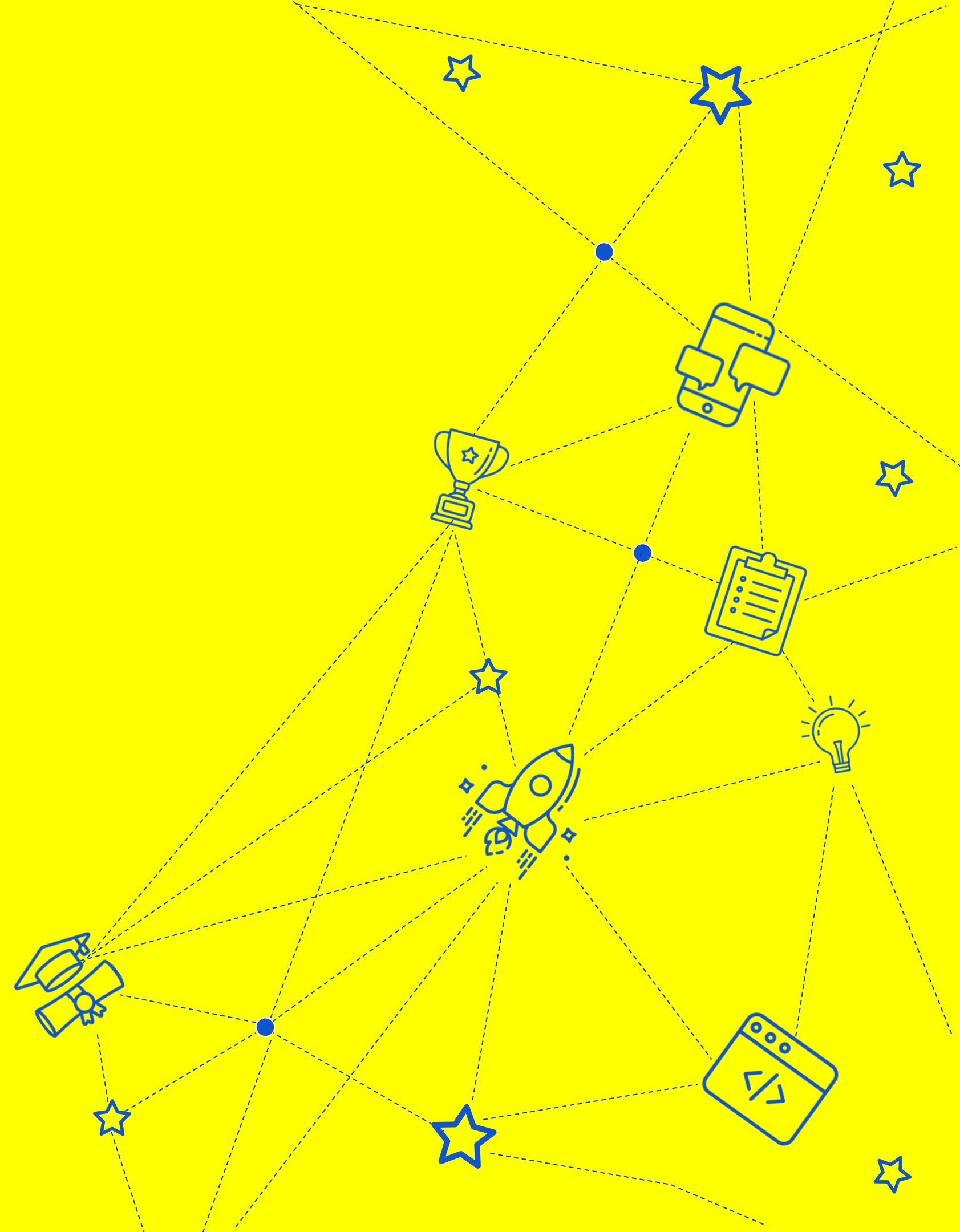
```
Name: aleks Email: mail@alex.com ApprovedBy: ivan
Name: ivan Email: mail@ivan.com ApprovedBy: admin
Name: ivan Email: mail@ivan.com ApprovedBy: petr
```

Спасибо за внимание

Продолжение в следующем видео



Воркшоп



Содержание воркшопа

Напишем построчно пример кода, который реализует главные концепты ООП:

- ★ Инкапсуляцию

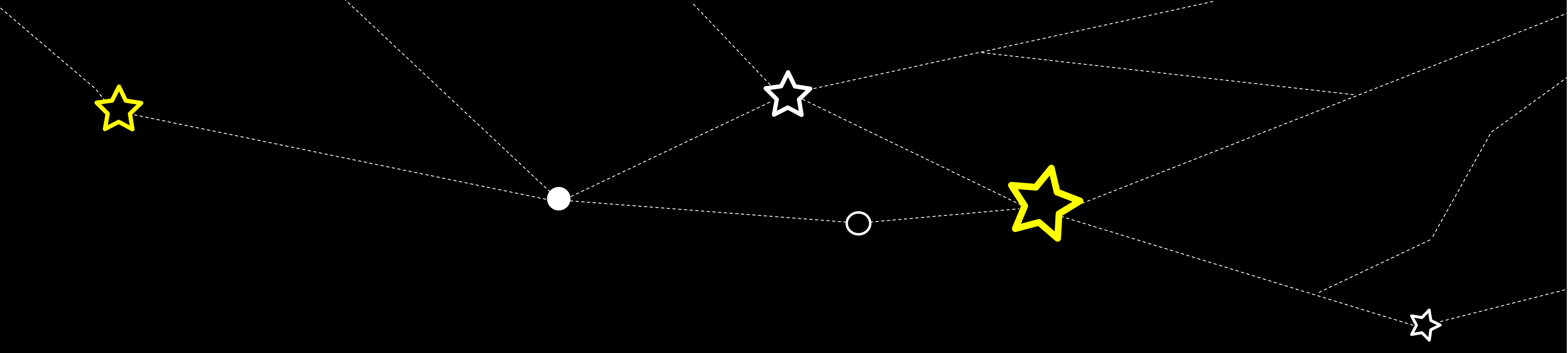
- ★ Наследование

- ★ Полиморфизм

А также, содержит:

- ★ Конструктор класса

- ★ Несколько методов класса



СПАСИБО ЗА ВНИМАНИЕ

Домашнее задание

На воркшопе мы разобрали пример кода с набором классов, унаследованных от класса User, в котором ведется работа с пользователями.

Дополните код классом **RegularUser**, представляющим собой класс обычного пользователя

